

```

import numpy as np
import math
import json
import csv
import matplotlib.pyplot as pyplot
with open("Simulation-settings.json","r") as json_file:
    settings = json.load(json_file)
    json_file.close()

class species:
    '''The species class represents the entire population of an species and
contains its base birth rate, death rate, current population, resources, name,
consumption rate, and it's population history.
    Additonaly resources expects a numpy array of numpy arrays, and population
expects a simple numpy array.
    "experiment_length" is a class variable, and is thus accesable to all instances
of this class. it tracks how long the experiment will go for, and is used to
establish the array that will note the population over the years
    "cycle_counter" is a class variable that tracks the ammount of cycles that
have occured so far'''
    experiment_length = settings.get("simulation duration")
    cycle_counter = 0
    species_population_dict = {}
    def __init__(self,birth_rate, death_rate,
population:np.ndarray,resources:np.ndarray =
np.array([0]),name:str="",consumption_rate:np.float64 = 1,nutritional_value:int=1):
        self.name = name
        self.birth_rate = birth_rate
        self.death_rate = death_rate
        self.population = population
        self.resources = resources
        self.consumption_rate = consumption_rate
        self.nutritional_value = nutritional_value
        self.population_history = np.zeros(self.experiment_length)
        self.population_history[0] = self.population[0]
        self.population_change_history = np.zeros(self.experiment_length)
    def overshoot_population_model(self):
        '''This is a modified version of the standard overshoot population model
that has been modified to not loop, and work within the context of the function
class.
        Thus this function can be called by
species_object.overshoot_population_model() instead of
overshoot_population_model(vars).
        Since this is a function within a class, calling it on the object will
automatically use the object with no propting due to the "self" argument'''
        # Variables
        #-----
        # self.birth_rate - base rate of new births per year
        # self.death_rate - base rate of deaths per year
        # self.consumption_rate - resources used by each organism of the species in

```

```

each year
    # self.population - current population of this species of organisms
    # self.resources - array of the resources available to this species of
organisms, with each entry in the array representing the population of another
species this species preys on
    # current_reasource_total - the total amount of resources available to this
species
    # shortage_death_rate - an additional deathrate representing the additional
amount of deaths during reasource scarcity
    # population_change - represents the total change in population
    # resources_ratio - tracks the percentage make of the different resources in
current_reasource_total
    # all_remainders - tracks the remainders, and the associated position of
it's associated number in the reasources array

    # Checks to see if population is 0, if it is we can skip all the future
calculations, and return 0 if applicable
    if self.population == 0:
        return 0
    # Use the sum function to sum all the values of the self.resources array
together giving current_reasource_total
    current_reasource_total = 0
    resource_species_list = np.empty(len(self.resources), dtype=species)
    for i in range(0, len(self.resources), 1):
        resource_species_list[i] =
self.species_population_dict[self.resources[i]]
        current_reasource_total += (resource_species_list[i].population[0] *
resource_species_list[i].nutritional_value)

    # creates an array to store the remainders later and using the dtype
argument creating a custom datatype so it can be sort in the order of remainders
datatype instead of ordering each element individually. (both datatypes are using
int64 as a base, indicated by the "i8")
    all_remainders =
np.zeros((len(self.resources),), dtype=[("remainders", "i8"), ("position", "i8")])

    # resources_per_person is the relationship between the two reservoirs
    if type(current_reasource_total) == np.ndarray:
        current_reasource_total = current_reasource_total[0]
    resources_per_person = current_reasource_total/self.population[0]

    # if resources_per_person is sufficiently low enough, impose an additional
death rate
    if resources_per_person/self.consumption_rate <= 10:
        shortage_death_rate = -0.1 *
(resources_per_person/self.consumption_rate) + 1.0
    else:
        shortage_death_rate = 0

    # Calculate the number of births that will happen with this current

```

```

population
    if resources_per_person/self.consumption_rate <= 50:
        #births = (self.birth_rate *
self.population[0])*(resources_per_person*0.02)
        pass
    else:
        pass
    births = self.birth_rate * self.population[0]

    # Calculate the number of deaths that will happen with this current
population
    deaths = (self.death_rate+shortage_death_rate) * self.population[0]

    # Calculate the total change in population
    population_change = births - deaths

    # How many resources were used up? 1 resource for each person
    resources_used = self.consumption_rate * self.population[0]

    # Apply the population change
    self.population[0] += round(population_change)

    # Population can't be less than zero!
    if self.population[0] < 0:
        self.population[0] = 0

    # Record the population
    self.population_history[self.cycle_counter] = self.population[0]

    # Record the change history
    self.population_change_history[self.cycle_counter] = population_change
    # Calculate what percentage a type of reasources make up the total current
resources
    remainders_sum = 0
    for i in range(0,len(self.resources),1):
        if resource_species_list[i].population > 0: # If statment for the value
not bein 0 to prevent div by 0 error
            resource_ratio = (resource_species_list[i].population[0] *
resource_species_list[i].nutritional_value)/current_reasource_total
        else:
            resource_ratio = 0 # If the else statment is false, then there must
be 0 resources, and as there are 0 resources, there is no way to contribute to the
ratio. Thus ratio is set to 0
        # Calculate the change in resources by multiplying the resources used
by the fractional resource ratio
        resource_species_list[i].population[0] -= (resources_used *
resource_ratio)/resource_species_list[i].nutritional_value
        # Caluclate the remainder of divison by 1 so that we know the numbers
after the decimal point

```

```

        remainder = resource_species_list[i].population[0] % 1.0
        # Store this remainder and it's index to the all_remainders array
        all_remainders[i] = remainder,i
        # Add the remainder to the growing sum of remainders
        remainders_sum += remainder

    # Round the remainders_sum
    remainders_sum.round()
    remainders_sum = int(remainders_sum)
    all_remainders = np.sort(all_remainders,order="remainders")[:-1]
    # Round up a number of individual resource amounts based on the closest
interger to the remainder
    for i in range(0,remainders_sum,1):
        resource_species_list[all_remainders[i][1]].population[0] =
math.ceil(resource_species_list[all_remainders[i][1]].population[0])
        if resource_species_list[all_remainders[i][1]].population[0] <= 0:
            resource_species_list[all_remainders[i][1]].population[0] = 0
    # Round down all remaining resource values
    for i in range(remainders_sum,len(self.resources),1):
        resource_species_list[all_remainders[i][1]].population[0] =
math.floor(resource_species_list[all_remainders[i][1]].population[0])
        if resource_species_list[all_remainders[i][1]].population[0] <= 0:
            resource_species_list[all_remainders[i][1]].population[0] = 0
    decay = int(round(deaths*self.consumption_rate))
    return decay

def overshoot_population_model_plants_decomposers(self):
    '''This is a modified version of the standard overshoot population model
that has been modified to not loop, and work within the context of the function
class.

```

Thus this function can be called by
species_object.overshoot_population_model() instead of
overshoot_population_model(vars).

Since this is a function within a class, calling it on the object will
automatically use the object with no propting due to the "self" argument'''

```

    # Variables
    #-----
    # self.birth_rate - base rate of new births per year
    # self.death_rate - base rate of deaths per year
    # self.consumption_rate - resources used by each organism of the species in
each year
    # self.population - current population of this species of organisms
    # self.resources - array of the resources available to this species of
organisms, with each entry in the array representing the population of another
species this species preys on
    # current_reasource_total - the total amount of resources available to this
species
    # shortage_death_rate - an additonal deathrate representing the additonal
amount of deaths during reasource scarcity
    # population_change - represents the total change in population
    # resources_ratio - tracks the percentage make of the diffrent resources in

```

```

current_reasource_total
    # all_remainders - tracks the remainders, and the associated position of
    it's associated number in the reasources array

    # Checks to see if poupluation is 0, if it is we can skip all the future
    calculations, and return 0 if applicable
    if self.population[0] == 0:
        if self.name == "plants":
            return
        else:
            return 0
    # Use the sum function to sum all the values of the self.resources array
    together giving current_reasource_total
    current_reasource_total = 0
    for i in range(0,len(self.resources),1):
        current_reasource_total += self.resources[i][0]

    # creates an array to store the remainders later and using the dtype
    argument creating a custom datatype so it can be sort in the order of remainders
    datatype instead of ordering each elmenet individually. (both datatypes are using
    int64 as a base, indicated by the "i8")
    all_remainders =
    np.zeros((len(self.resources),),dtype=[("remainders","i8"),("position","i8")])

    # resources_per_person is the relationship between the two reservoirs
    resources_per_person = current_reasource_total/self.population[0][0]

    # if resources_per_person is sufficently low enough, impose an additonal
    death rate
    if resources_per_person/self.consumption_rate <= 10:
        shortage_death_rate = -0.1 *
        (resources_per_person/self.consumption_rate) + 1.0
    else:
        shortage_death_rate = 0

    # Calculate the number of births that will happen with this current
    population
    if resources_per_person/self.consumption_rate <= 10:
        births = (self.birth_rate *
        self.population[0][0])*(resources_per_person*0.1)
    else:
        births = self.birth_rate * self.population[0][0]

    # Calculate the number of deaths that will happen with this current
    population
    deaths = (self.death_rate+shortage_death_rate) * self.population[0][0]

    # Calculate the total change in population
    population_change = births - deaths

```

```

# How many resources were used up? 1 resource for each person
resources_used = self.consumption_rate * self.population[0][0]

# Apply the population change
self.population[0][0] += round(population_change)

# Population can't be less than zero!
if self.population[0][0] < 0:
    self.population[0][0] = 0

# Record the population
self.population_history[self.cycle_counter] = self.population[0][0]

# Record the change history
self.population_change_history[self.cycle_counter] = population_change
# Calculate what percentage a type of reasources make up the total current
resources
remainders_sum = 0
for i in range(0, len(self.resources), 1):
    if self.resources[i][0] != 0: # If statment for the value not bein 0 to
prevent div by 0 error
        resource_ratio = self.resources[i][0]/current_reasource_total
    else:
        resource_ratio = 0 # If the else statment is false, then there must
be 0 resources, and as there are 0 resources, there is no way to contribute to the
ratio. Thus ratio is set to 0
    # Calculate the change in resources by multiplying the resources used
by the fractional resource ratio
    self.resources[i][0] -= resources_used * resource_ratio
    # Caluclate the remainder of divison by 1 so that we know the numbers
after the decimal point
    remainder = self.resources[i][0] % 1.0
    # Store this remainder and it's index to the all_remainders array
    all_remainders[i] = remainder, i
    # Add the remainder to the growing sum of remainders
    remainders_sum += remainder

# Round the remainders_sum
remainders_sum.round()
remainders_sum = int(remainders_sum)
all_remainders = np.sort(all_remainders, order="remainders")[:-1]
# Round up a number of individual resource amounts based on the closest
integer to the remainder
for i in range(0, remainders_sum, 1):
    self.resources[all_remainders[i][1]][0] =
math.ceil(self.resources[all_remainders[i][1]][0])
    if self.resources[all_remainders[i][1]][0] <= 0:
        self.resources[all_remainders[i][1]][0] = 0
# Round down all remaing resource values

```

```

        for i in range(remainders_sum, len(self.resources), 1):
            self.resources[all_remainders[i][1]][0] =
math.floor(self.resources[all_remainders[i][1]][0])
            if self.resources[all_remainders[i][1]][0] <= 0:
                self.resources[all_remainders[i][1]][0] = 0
            if self.name == "plants":
                return
            elif self.name == "decomposers":
                return round(resources_used*self.consumption_rate)
# Begin importing settings settings from the json file
with open("Input-data.csv", "r") as csv_file:
    data = csv.reader(csv_file)
    csv_file.close
    n = 0
    species_list = []
    for row in data:
        if n >= 1:
            species_holder =
species(float(row[1]), float(row[2]), np.array([np.float64(row[3])]), resources=np.arr
ay(row[4].split("&")), name=row[0], consumption_rate=np.float64(row[5]), nutritional_v
alue=int(row[6]))
            species_list.append(species_holder)
        n += 1

# create dictionary and store species objects and associated names
species_name_dictionary = {}
for n in range(0, len(species_list), 1):
    species_name_dictionary[species_list[n].name] = species_list[n]

# begin applying settings to hard coded vars, all of these settings are in the form
of dictionaries
initial_corpses = np.array([settings.get("decomposers settings").get("initial
resources")])
initial_plant_food = np.array([settings.get("plants settings").get("initial
resources")])
plant_reproduction_rate = settings.get("plants settings").get("reproduction rate")
plant_death_rate = settings.get("plants settings").get("death rate")
initial_plants = np.array([settings.get("plants settings").get("initial
population")])
plant_nutritional_value = settings.get("plants settings").get("nutritional value")
decomposers_reproduction_rate = settings.get("decomposers
settings").get("reproduction rate")
decomposers_death_rate = settings.get("decomposers settings").get("death rate")
initial_decomposers = np.array([settings.get("decomposers settings").get("initial
population")])
plant_consumption_rate = settings.get("plants settings").get("consumption rate")
decomposers_consumption_rate = settings.get("decomposers
settings").get("consumption rate")
decomposers_nutritional_value = settings.get("decomposers
settings").get("nutritional value")

```

```

do_removal_flag = settings.get("removal settings").get("do removal")
do_removal_point = settings.get("removal settings").get("removal point")
removal_target = settings.get("removal settings").get("removed species")
removal_percentage = settings.get("removal settings").get("removal percent")
tracked_species = settings.get("requested data").get("select species")
# end settings application

# initiate the hardcoded decomposers and plants
decomposers =
species(decomposers_reproduction_rate,decomposers_death_rate,np.array([initial_deco
mposers],float),np.array([inital_corpses],float),"decomposers",decomposers_consumpt
ion_rate,decomposers_nutritional_value)
plants =
species(plant_reproduction_rate,plant_death_rate,np.array([inital_plants],float),np
.array([inital_plant_food],float),"plants",plant_consumption_rate,plant_nutritional
_value)

# add decomposers and plants to the species dictionary
species_name_dictionary[decomposers.name] = decomposers
species_name_dictionary[plants.name] = plants

# initilise ecosystem
ecosystem = np.array(species_list)

# add dictionary
species.species_population_dict = species_name_dictionary

# delete unnecessary values
del initial_corpses, initial_plant_food, inital_plants, initial_decomposers, row,
settings, data, decomposers_nutritional_value, plant_nutritional_value,
species_holder

# the loop for the ecosystem
for i in range(0,species.experiment_length-1,1):
    species.cycle_counter += 1
    plants.overshoot_population_model_plants_decomposers()
    for x in range(0,len(ecosystem),1):
        corpses = ecosystem[x].overshoot_population_model()
        decomposers.resources[0][0] += corpses
        plants.resources[0][0] +=
    decomposers.overshoot_population_model_plants_decomposers()
    if do_removal_flag and do_removal_point == i:
        species_name_dictionary[removal_target].population[0] -= removal_percentage
* species_name_dictionary[removal_target].population[0]
    timeline = np.arange(0,species.experiment_length,1)
    plotted_species_array = np.zeros(len(tracked_species))

# Begin plotting
pyplot.figure(figsize=(10,6))

```

```

for i in range(0,len(tracked_species),1):

pyplot.plot((species_name_dictionary[tracked_species[i]].population_history),label
= tracked_species[i])
pyplot.xlabel("Time in weeks")
pyplot.ylabel("Population")
pyplot.yscale("log")
pyplot.legend(title = "species")
pyplot.title(f'Population of various species in an ecosystem over
{species.experiment_length} weeks, with {100*removal_percentage}% snakes removed at
{do_removal_point} weeks')

# export plot
pyplot.savefig(f'{100*removal_percentage} percent removed.png')

```

JSON section

```

{
  "simulation duration" : 500,
  "removal settings":{
    "do removal": true,
    "removal point" : 20,
    "removal percent": 1.0,
    "removed species": "puff adders snake"
  },
  "decomposers settings":{
    "reproduction rate" : 0.4,
    "death rate" : 0,
    "inital population": 10000,
    "inital resources": 10000000,
    "consumption rate": 0.8,
    "nutritional value": 1
  },
  "plants settings":{
    "reproduction rate" : 0.4,
    "death rate" : 0,
    "inital population": 100000,
    "inital resources": 100000000,
    "consumption rate": 0.4,
    "nutritional value": 1
  },
  "requeusted data":{
    "select species": ["striped mice","puff adders snake","brown snake eagle"],
    "select data": "population"
  }
}

```

CSV section

name,birth rate,death rate,population,resources,consumption rate,nutritional value

"striped mice",0.6329,0.0196,500,"plants",0.0192,1
"puff adders snake",0.2877,0.0016,200,"striped mice",0.8865,6
"brown snake eagle",0.0096,0.0019,150,"striped mice&puff adders snake",1.3298,9